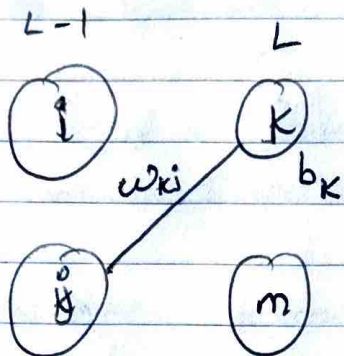


①

Neural networks : complete + from the start

a neuron stores a value and that value can be fed through the network, this value is called the activation value, a



$$a_i^L = f(z_i^L) \quad (1)$$

where f is some function which "squishes" z_i^L values

acc → to
from → acc
b₀ → from

feed forward

$$z_i^L = \sum_j w_{ij}^L a_j^{L-1} + b_i^L \quad (2)$$

↑
 i^{th} neuron
in L -layer

↑
to i^{th} neuron in L
from j^{th} neuron in $L-1$

(1) and (2)

this updates the activation of each neuron in each layer by feeding forwards. We then implement a cost function to evaluate how good our initial guesses at w_{jk} and b_j were. Some examples include.

quadratic cost

Cross-entropy-cost

$$C = \frac{1}{2} |y_j - a_j^L|^2 \quad (3)$$

$$C = -y_i \ln a_i - (1-y_i) \ln(1-a_i) \quad (4)$$

y_j is the correct output for the j^{th} neuron, i.e. 1 if correct zero if not

this eliminates learning slowdown issues caused by quadratic cost, these will be mentioned later

(2)

there are other cost functions we will evaluate later like log-likelihood but now we have the cost of our layer for a single training example we want to minimise this cost. We do this via gradient descent. How small changes to weights and biases change the cost.

The gradient of a function shows the steepest increase $\therefore$
 $-\nabla$ shows the steepest decrease as desired.

We don't have info on the individual weights and biases but we do know the final activations so we can develop formulae which propagate back from this layer

backpropagation

we define an error variable δ_j^l :

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} \quad \text{for any layer / any neuron}$$

by chain rule

$$= \frac{\partial C}{\partial a_j^l} \frac{\partial a_j^l}{\partial z_j^l}$$

recalling (1)

$$= \frac{\partial C}{\partial a_j^l} \cdot f'(z_j^l)$$

This is for a single neuron, to calculate a vector of errors for the layer,

$$\delta^L = \nabla_a C \odot f'(z^L) \quad (5)$$

This is summed over all neurons in layer L

$\odot$ is vector elementwise multiplication

3

This is easy to calculate as we have the activations of neurons in the final layer. We now want a recursive formula to tell us δ^l for other layers.

$$\delta_j^{l+1} = \frac{\partial C}{\partial z_j^{l+1}}$$

recalling (2)

$$\delta_j^{l+1} = \frac{\partial C}{\partial z_k^l} \frac{\partial z_k^l}{\partial z_j^{l+1}}$$

$$z_j^l = w_{jk}^l a_k^{l-1} + b_j^l$$

$$z_j^l = w_{jk}^l \sigma(z_k^{l-1}) + b_j^l$$

$$\delta_j^{l+1} = \delta_k^l \frac{\partial z_k^l}{\partial z_j^{l+1}}$$

B.N. indices are switched but are only dummy so irrelevant

$$\delta_j^{l+1} = \delta_k^l \cdot w_{jk}^l \sigma'(z_k^{l-1})$$

this is for a single neuron k to j for whole layer

$$\delta^{L-1} = (w_{in}^L)^T \delta^L \odot \sigma'(z^{L-1}) \quad (6)$$

transpose of weight matrix required so that dimensionally it can be multiplied by δ^L vector.

now we know δ^L and how to find δ^{L-1} we can finally find $\partial C / \partial w$ and $\partial C / \partial b$ and find how they need to be altered

$$\frac{\partial C}{\partial w_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{jk}^l}$$

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l}$$

$$\frac{\partial C}{\partial w_{jk}^l} = \delta_j^l a_k^{l-1} \quad (7)$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \quad (8)$$

(5), (6), (7) and (8) form the fundamental equations of backpropagation

④

So we can finally get the gradient for weights and biases for a single training example, represented in a vector.

But computing the gradient for every single training example is an enormous task so instead we do stochastic gradient descent.

We take mini-batches of a small size, m , and average the gradient over these m samples, then update the weights + biases and run another batch. We update the weights + biases as follows...

$$\omega' = \omega - \frac{1}{m} \sum_x \frac{\partial \mathcal{L}_{x_i}}{\partial \omega_{jk}^l} \quad (9)$$

$$b' = b - \frac{1}{m} \sum_x \frac{\partial \mathcal{L}_{x_i}}{\partial b_j^l} \quad (10)$$

what is the $f(z_j^l)$ mentioned earlier - could be any function with range that maps inputs somehow. Common one ReLU, $\tanh(x)$ and sigmoid is most common. Let's explore sigmoid and see the problem it exposes with quadratic cost

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (11)$$

$$\sigma'(z) = \frac{-e^{-z}}{(1 + e^{-z})^2}$$

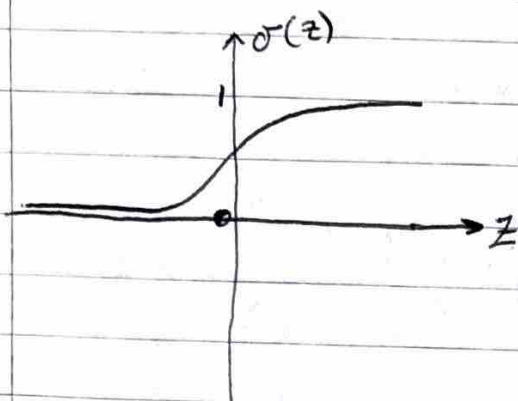
$$\sigma'(z) = \frac{e^{-z} + 1 - 1}{(1 + e^{-z})^2}$$

$$\sigma'(z) = \frac{1 + e^{-z}}{(1 + e^{-z})^2} - \frac{1}{(1 + e^{-z})^2}$$

$$\sigma'(z) = \sigma(z) [1 - \sigma(z)] \quad (12)$$

⑤

Let's look at the shape of the sigmoid to see why it causes a problem with quadratic cost.



This sketch of sigmoid shows $\sigma'(z) \rightarrow 0$ as $z \rightarrow \pm \infty$

with quadratic cost //

$$\begin{aligned} \delta_j^l &= \frac{\partial C}{\partial z_j^l} \\ &= \frac{\partial C}{\partial a_j^l} \sigma'(z_j^l) \\ &= [a_j^l - y_j] \cdot \sigma'(z_j^l) \quad (13) \end{aligned}$$

So when the activation of the neuron is close to 0 or 1, the corresponding $\sigma'(z_j^l)$ will be ≈ 0 so the δ_j^l and thus $\frac{\partial C}{\partial w}$ will also be very small. The consequence is that these neurons are saturated, slow to learn and the model learns the least from its biggest mistakes - this is learning slowdown.

This can be rectified by using a cross-entropy cost such that...

$$C = -y_i \ln a_i - (1 - y_i) \ln (1 - a_i) \quad (4)$$

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} = \frac{\partial}{\partial z_j^l} \left[-y_i \ln \sigma(z_i) - (1 - y_i) \ln (1 - \sigma(z_i)) \right]$$

noting that y_i is constant.

(6)

$$\delta_j^l = -\frac{y_i}{\sigma(z_i)} \frac{\partial \sigma}{\partial z} + \frac{(1-y_i)}{1-\sigma(z_i)} \frac{\partial \sigma}{\partial z}$$

$$\delta_j^l = \frac{\partial \sigma}{\partial z} \left[\frac{-y_i(1-\sigma(z_i)) + \sigma(z_i)(1-y_i)}{\sigma(1-\sigma)} \right]$$

recall
(12)

$$\delta_j^l = \frac{\sigma(1-\sigma)}{\sigma(1-\sigma)} (-y_i + \sigma(z_i))$$

$$\delta_j^l = a_j^l - y_j \quad (14)$$

δ_j^l is no longer dependent on $\sigma'(z_i^l)$ so cross-entropy prevents learning slow down. Then equations (5), (6), (7), (8) can be used to backpropagate.

Another solution to the learning slowdown is to use a function called softmax. We feed through the network and then with the final layer of z values we do the following.

$$a_j^L = \frac{e^{z_j^L}}{\sum_i e^{z_i^L}} \quad (15)$$

This essentially gives a weighted sum and a probability distribution of activations

Sum over
all neurons
in that layer

$$\sum_j a_j^L = \sum_j \frac{e^{z_j^L}}{\sum_i e^{z_i^L}}$$

Proof that softmax gives a probability distribution:

$$= \frac{1}{\sum_i e^{z_i^L}} \cdot \sum_j e^{z_j^L}$$

i, j are dummy variables
so sums over them are
identical

$$= 1 \quad (16)$$

⑦

As always we are interested in σ_j^L , $\frac{\partial C}{\partial z_i^L}$ and thus $\frac{\partial \sigma_j^L}{\partial z_i^L}$ as $\frac{\partial a_j^L}{\partial z_i^L}$ but softmax presents two cases:

- ① the z_j we differentiate with respect to has a_i such that $i=j$
- ② has a_i such that $i \neq j$

for case of $i=j$ // letting $\sum_i e^{z_i^L} = S$

$$\begin{aligned} \frac{\partial a_j}{\partial z_i} &= \frac{\partial}{\partial z_i} \left[\frac{e^{z_j}}{S} \right] \quad \text{as } i=j \text{ differentiation preserves } e^{z_j} \\ &= \frac{S \cdot e^{z_j} - e^{z_j} e^{z_j}}{S^2} \\ &= \frac{e^{z_j}}{S} \left[\frac{S - e^{z_j}}{S} \right] \\ &= a_j (1 - a_j) \quad (17) \end{aligned}$$

for $i \neq j$

$$\begin{aligned} \frac{\partial a_j}{\partial z_i} &= \frac{\partial}{\partial z_i} \left[\frac{e^{z_j}}{S} \right] \quad \text{as } i \neq j \text{ no preservation} \\ &= \frac{S \cdot 0 - e^{z_j} e^{z_i}}{S^2} \\ &= -a_j a_i \quad (18) \end{aligned}$$

now we have these derivatives we relate it to a cost function called log-likelihood:

$$C \equiv -\log_{10}^L - \ln a_j^L \quad (19)$$

8

This means the cost depends only on the activation of the correct class neuron. i.e. if the image is of a 7, we want the activation of a_7 to be ≈ 1 , but as $a_7 \rightarrow 0$ $-\ln a_7 \rightarrow \infty$ as required by a good cost function.

for loglikelihood //

$$\frac{\partial C}{\partial z_j^l} = \frac{\partial C}{\partial a_j^l} \frac{\partial a_j^l}{\partial z_j^l}$$

$$\hat{y} = j$$

$$i \neq j$$

$$\sigma_j^l = -\frac{1}{a_y} [a_y(1-a_y)]$$

$$\sigma_j^l = -\frac{1}{a_y} \cdot a_y a_j$$

$$= a_y - 1$$

$$= a_j$$

$$(20) \sigma_j^l = a_j - y_i \quad \text{where } y_i = \begin{cases} 0 & i \neq j \\ 1 & i = j \end{cases}$$

Again no σ' so learning slowdown problem is solved.

A new problem emerges when you train networks for too long - they begin to memorise the training data instead of generalising to test data - so even though cost continues to decrease accuracy tanks.

We fix this problem through regularization of the cost function.

L2 //

$$C = C_0 + \frac{1}{2n} \sum_i w_i^2 \quad (21)$$

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{1w}{n}$$

where this is a sum over all weights and C_0 is some cost function like quadratic or cross entropy

We then implement this into our weight modification formula (9)

(9)

$$\omega' = \omega - \eta \left(\frac{\partial C_0}{\partial \omega} + \frac{\lambda \omega}{\eta} \right)$$

$$\omega' = \underbrace{\omega \left(1 - \frac{\lambda \eta}{\eta} \right)} - \eta \frac{\partial C_0}{\partial \omega} \quad (22)$$

This is the weight decay term and it shrinks the weights and makes memorisation of training data more difficult. Other methods like "dropout" are interesting. Biases are not regularized as they don't cause overfitting.

momentum implemented //

We implement a recursive formula such that

$$\omega' = \omega + v \quad (23)$$

$$v_{t+1} = \mu v_t - \eta \nabla C$$

First thing to notice is that if $\mu=0$ (23) reverts back to a formula we have long used. But let's explore how the recursion iterates.

$$v_{t+1} = \mu v_t - \eta \nabla C_t$$

$$v_{t+1} = \mu (\mu v_{t-1} - \eta \nabla C_t) - \eta \nabla C_t$$

each velocity is an exponentially weighted moving average of all previous gradients. Choosing a higher μ value gives the velocity a longer memory. It helps by doing the following...

- ① if gradient is consistently in one direction amplifies it to more quickly approach minimum
- ② if gradient is going through a narrow valley it damps oscillation in the valley because the weighted sum cancels flipping signs

(10)

This approach is perfectly fine for shallow networks, but it begins to fail for deep networks which have the potential to solve much more complex problems. This failure occurs because of gradient instability. It arises because of eqn (6)...

$$\delta^{l-1} = (w^l)^T \delta^l \sigma'(z^{l-1})$$

as we iterate over many δ 's for multiple layers the further we propagate back the more $(w^l)^T \delta^l \sigma'(z^{l-1})$ terms we have. Given $\max(\sigma'(x)) = \frac{1}{4}$ and $w^l \lesssim 1$ at initialisation earlier layers get very dulled and learn very slowly as they have a vanishing gradient. if the weights are initialised too large the opposite problem of exploding gradient can occur but either way we need a solution to be able to train deep neural nets.